

Software Requirements Specification (SRS)

Project: Offline-First, Proxy-Free College Attendance System

Version: 1.1 Date: August 2026 Prepared by: Quad Shell



1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for a highly secure, offline-first college attendance tracking system. It details the functional and non-functional requirements to serve as a blueprint for the development team and a contract for project evaluation.

1.2 Scope

Traditional attendance systems suffer from proxy marking (via screenshots, remote links, or multiple devices) and system failures during classroom internet blackouts. This system solves these issues using Cryptographic Time-Based One-Time Passwords (TOTP), on-device biometric gaze verification, hardware-backed WebAuthn binding, and offline local queuing. It is designed to work efficiently even in large 120-240 seat lecture halls with zero network connectivity.

1.3 Definitions and Acronyms

- **PWA:** Progressive Web Application.
- **TOTP:** Time-Based One-Time Password.
- **WASM:** WebAssembly (used for running client-side machine learning models).
- **IndexedDB:** A low-level API for client-side storage of structured data.
- **WebAuthn:** Web Authentication API used for hardware-level device binding.
- **Nonce:** A cryptographic number used only once to prevent replay attacks.
- **BSSID:** Basic Service Set Identifier (MAC address of a wireless access point).

2. Overall Description

2.1 Product Perspective

The system operates as a distributed web application with three main components:

1. A mobile PWA for students.
2. A laptop web interface for faculty (projector display).
3. A central Python/FastAPI and PostgreSQL backend for synchronization, real-time web sockets, administration, and data persistence.

2.2 User Classes and Characteristics

- **Student:** Uses a mobile smartphone. Requires a low-friction UI, offline capabilities, and instant feedback.

- **Faculty/Professor:** Uses a laptop connected to a projector. Needs a reliable interface to display QR codes, manage rosters, query records, and export data.
- **Administrator:** Department coordinator who manages course mapping, student enrollment, and hardware reset approvals.

2.3 Operating Environment

- **Client (Student):** Modern mobile browsers (Chrome, Safari, Firefox) supporting HTML5, WebGL, WebRTC (Camera), WebAuthn, and Service Workers.
- **Client (Faculty):** Standard desktop browsers connected to large hall projectors.
- **Server:** Python FastAPI runtime, PostgreSQL relational database, hosted on a standard cloud provider.

2.4 Assumptions and Dependencies

- Students have access to a smartphone with a functioning rear and front camera.
- Faculty laptops have their internal system clocks synchronized to standard internet time (UTC) prior to going offline.
- Students will open the PWA at least once with an internet connection to cache core assets (Cold Start).

3. System Features (Functional Requirements)

3.1 Student Module (Mobile PWA)

- **REQ-S1: Zero-Install PWA Access.** The system shall run in mobile browsers and cache assets via Service Workers for offline use.
- **REQ-S2: Hardware Device Binding (WebAuthn).** Upon first login, the system shall cryptographically bind the student's profile to the device's hardware security chip (Secure Enclave) to prevent account hopping.
- **REQ-S3: QR Scanner Integration.** The app shall utilize the device's rear camera to scan dynamic QR codes projected in the classroom.
- **REQ-S4: On-Device Biometric Face Matching.** The app shall run a WASM-based model (e.g., MediaPipe) to match live camera feed against a pre-registered 128D facial vector.
- **REQ-S5: Active Gaze Liveness Challenge.** The app shall require the user to visually track a randomly moving on-screen dot to defeat photo/video spoofing.
- **REQ-S6: Offline Storage Queue.** If internet is unavailable, the system shall encrypt and save the verified scan payload, including the embedded session timestamp, into `IndexedDB`.
- **REQ-S7: Background Auto-Sync.** The PWA shall automatically transmit queued offline records to the server once network connectivity is restored.
- **REQ-S8: Device Reset Request.** Students shall be able to submit a justification workflow to admins if their bound device is lost or replaced.

3.2 Faculty Module (Laptop Web App)

- **REQ-F1: Offline Dynamic TOTP Engine.** The laptop app shall compute and display a cryptographic QR token that refreshes every 5 seconds, requiring zero active internet connection.
- **REQ-F2: Projector Optimization Mode.** The UI shall project high-contrast, scalable QR codes optimized for distant scanning in large lecture halls (optical line-of-sight constraint).
- **REQ-F3: Live Attendance Counter.** The dashboard shall display a real-time count of synced attendance records via FastAPI WebSockets.
- **REQ-F4: Manual Override Panel.** The faculty shall have a toggle to manually mark students present who experience hardware/camera failures.
- **REQ-F5: Direct Local Export.** Faculty shall be able to download session logs as `.xlsx` or `.csv` files directly from the dashboard.
- **REQ-F6: Optional Google Sheets Sync.** Faculty shall have the option to link a `Spreadsheet ID` to batch-sync attendance records via Google Sheets API.
- **REQ-F7: Advanced Search and Filtering.** Faculty shall be able to query historical attendance records by multiple parameters: Date range, specific Student Roll Number, Student Name, or specific Class/Session IDs.
- **REQ-F8: Overall Attendance Analytics.** The faculty dashboard shall calculate and display overall attendance percentages, flagging students who fall below the minimum required threshold (e.g., 75%).

3.3 Administrator Module

- **REQ-A1: CSV/Excel Bulk Onboarding.** Admins shall be able to upload bulk rosters for students, faculty, and courses.
- **REQ-A2: Reset Approval Workflow.** Admins shall review and approve/reject student device hardware reset requests.
- **REQ-A3: Audit Logs.** The system shall provide logs of suspicious activities, including hardware-mismatch logins and duplicate nonce submissions.

4. Non-Functional Requirements

4.1 Performance & Scalability

- **PER-1:** The client-side biometric gaze verification must compute in less than 2.5 seconds on standard budget smartphones.
- **PER-2:** The backend batch-sync worker must handle database commits without rate-limiting external APIs (e.g., Google Sheets).
- **PER-3:** The FastAPI backend must validate and route incoming JSON payloads in under 100ms.

4.2 Security & Fraud Prevention

- **SEC-1:** System shall strictly reject tokens older than the 5-second active window to prevent screenshot sharing.
- **SEC-2:** The server must validate the embedded professor's timestamp inside the payload, completely ignoring the student device's local clock to prevent time-tampering.

- **SEC-3:** All payloads submitted to the server must be cryptographically signed (HMAC) to prevent client-side JavaScript tampering via browser DevTools.
- **SEC-4:** Replay attacks shall be mitigated by validating single-use nonces and composite database keys (`session_id` + `student_id`).

4.3 Reliability & Availability

- **REL-1:** The system must guarantee zero data loss during total classroom network blackouts, retaining records locally in `IndexedDB` for at least 7 days if necessary.
- **REL-2:** The Google Sheets integration must be loosely coupled; if the Google API is down, the system continues to function normally using the primary PostgreSQL database.

5. External Interface Requirements

5.1 User Interfaces

- **Student UI:** Minimalist, camera-first interface showing scan status, offline sync queue status, and overall attendance statistics.
- **Faculty UI:** Dashboard with prominent "Start Session" controls, large central QR display area, comprehensive search/filter data tables, and a sidebar for manual overrides.

5.2 Software & API Interfaces

- **FastAPI (OpenAPI/Swagger):** Used for automated interactive API documentation and strict data validation.
- **Google Sheets API v4:** Used for optional batch export of verified attendance logs.
- **Web Media API (HTML5):** Used to access the smartphone camera directly from the browser.
- **WebAssembly (WASM):** Used to execute client-side machine learning models natively in the browser without server communication.